

Zunaira Abdul Aziz

BSSE23058

Assignment 3

Q1. Explain why the ReLU (Rectified Linear Unit) activation function is generally preferred over Sigmoid for the hidden layers of deep neural networks. Specifically, discuss how each handles the "vanishing gradient" problem during backpropagation.

The ReLU is generally preferred over Sigmoid for hidden layers because Sigmoid functions saturate at the extremes (0 and 1), causing gradients to become very small.

Handling the Vanishing Gradient problem:

- **Sigmoid:** During backpropagation, gradients are calculated using the chain rule. The derivative of the Sigmoid function is always between 0 and 0.25. When many layers are stacked, multiplying these small derivatives causes the gradient to decay exponentially towards zero (vanish) as it propagates back to earlier layers, stopping them from learning.
- **ReLU:** The derivative of ReLU is either 0 (for negative inputs) or 1 (for positive inputs). Because the gradient is exactly 1 for positive inputs, it does not saturate or diminish as it passes through many layers, effectively mitigating the vanishing gradient problem.

Q2. You are training a deep network and observe that your Training Accuracy is 98% but your Validation Accuracy is only 72%.

A) What is this phenomenon called?

Overfitting. The model has memorized the training data (98%) but fails to generalize to unseen data (72%).

B) Name two specific techniques you could add to the architecture or training process to reduce this gap.

To reduce this gap, you could implement:

1. **Dropout:** Randomly deactivating neurons during training to prevent co-adaptation.
2. **Regularization:** Adding a penalty term to the loss function to discourage large weights.

Q3. Consider a single fully connected (Dense) layer with 4 input neurons and 3 output neurons.

• **Calculate the total number of learnable parameters (weights + biases) for this layer.**

• **Hint: Don't forget the bias term for each output neuron.**

- Weights: 4 inputs*3 outputs = 12 weights
- Biases: = 3 biases(output=biases)

Total Parameters: 12 weights + 3 biases = 15 para

Q4. CNN Dimensions

You have an input image of size $32 \times 32 \times 3$ (RGB). You apply a Convolutional Layer with the following hyperparameters:

• **Filter (Kernel) Size:** 5×5 , **Stride:** 1, **Padding:** 0, **Number of Filters:** 10

Calculate the spatial dimensions (Height $\times$ Width) and the depth (channels) of the output feature map.

$$W_{out} = \left\lfloor \frac{W_{in} - K + 2P}{S} \right\rfloor + 1$$

$$[(32-5+2(0))/1]+1= 28$$

Final dimensions= $28 \times 28 \times 10$

Q5. In a typical CNN architecture (like AlexNet), Max Pooling layers are inserted between Convolutional layers. What are the two primary benefits of doing this regarding computational cost and feature invariance?

1. **Computational Cost:** It reduces the spatial dimensions (downsampling), which significantly decreases the number of parameters and computations required in subsequent layers
 2. **Feature Invariance:** It provides a degree of translation invariance, meaning the network can detect features (like an edge or texture) regardless of small shifts in position within the input image
-

Q6. Standard RNNs theoretically can remember long sequences, but in practice, they fail due to the Vanishing Gradient problem during Backpropagation Through Time (BPTT). Mathematically, what happens to the gradients as they are propagated back through many time steps (e.g.,

t100 → t1)?

As gradients are propagated back through many time steps, they involve the repeated multiplication of the recurrent weight matrix. If the weights (or eigenvalues of the weight matrix) are smaller than 1, the gradients shrink exponentially toward zero, preventing the network from learning long-range dependencies.

Q7. You are training a model on a dataset with very limited computational resources (memory and training time) and a relatively small amount of training data.

A) Would you choose an LSTM or a GRU?

GRU

B) Justify your choice by comparing the number of gates and the resulting parameter complexity of both architectures. Which specific gate does the GRU combine/remove compared to the LSTM?

Parameter Complexity: GRUs are structurally simpler than LSTMs. An LSTM has 3 gates (Input, Forget, Output), while a GRU has only 2 gates. This results in fewer learnable parameters, making the GRU faster to train and less memory-intensive, which fits the constraint of limited resources and small data.

Gate Operations: The GRU combines the Forget Gate and the Input Gate into a single Update Gate and removes the separate Output Gate entirely.

Q8. RNNs inherently understand the order of words because they process them one by one. Transformers process all words in parallel. Explain how a Transformer knows that "The cat ate the mouse" is different from "The mouse ate the cat" if it sees all words simultaneously.

Since Transformers process all words in parallel, they lack inherent order information. To solve this, Positional Encodings (vectors containing information about the position of each word) are added to the word embeddings at the input. This allows the model to distinguish that "The cat" (subject) is different from "The cat" (object) based on their positions in the sentence.

Q9. In the context of Self-Attention, we compute a score using Query (Q), Key (K), and Value (V) vectors. If Q represents "what I am looking for" and K represents "the label of the content," what mathematical operation (discussed in class) is used to calculate the similarity between Q and K?

What happens to the V (content) if this similarity score is close to zero?

Mathematical Operation: The operation used to calculate the similarity between the Query (Q) and the Key (K) is the Dot Product. The class slides explicitly state that the dot product (Q.K) "measures similarity" and determines how relevant one word is to another.

Effect on Value (V): If the similarity score is close to zero, the corresponding Value (V) is "**drowned out**". The mechanism is designed to "keep the important info" and drown out irrelevant information by multiplying the Value vector by the normalized score.

Q10. We discussed that BERT is an "Encoder-only" model, meaning it discards the Decoder component of the original Transformer. Please explain why?

BERT discards the Decoder because it is designed to create deep bidirectional representations of text. The original Transformer Decoder is "autoregressive" (masked), meaning it can only see previous words to generate the next one. By using only the Encoder, BERT can apply "Self-Attention" to the entire sequence at once (looking at both left and right context simultaneously), which is essential for understanding tasks like sentiment analysis or question answering, rather than text generation.
